TASK-005 - Efeitos de status e congelamento probabilístico

Spirit Gate | 18/10/2026 | Java 17 e Maven

Resultado

A Criostase oferece uma tentativa independente de 30% de Freeze a cada mob que receber dano positivo e sobreviver ao pulso. Um sucesso aplica ou renova a duração para exatamente 1 segundo, sem somar o tempo anterior. Fracassos não removem ou renovam o efeito. Mobs congelados não se movimentam, mas continuam podendo atacar em alcance.

Foram preservados os parâmetros da habilidade: dano base 7 ICE, custo 55 de vigor, cooldown 3 segundos, alcance 240, raio 85, duração 2,5 segundos e cinco pulsos em 0/0,5/1/1,5/2 s. Freeze não causa dano, não altera a frequência dos pulsos e não cria um sexto pulso.

Preparação e proposta

Antes das alterações foram inspecionados CryostasisService, CryostasisZone, CombatService, MobFactory, MovementService, DebugWorld, DebugSnapshot, DebugPanel e testes relacionados. Estado inicial: mvn clean test -q aprovado com 73 testes.

A proposta apresentada separou armazenamento por entidade, aplicação/atualização de efeitos e sorteios por um gerador injetável. O bloqueio fica no serviço autoritativo de movimento, e o snapshot publica uma coleção separada para preservar ActorView e o replay legado. Uma revisão paralela somente de leitura confirmou a ordem temporal e a preservação do ataque; nenhum achado bloqueador foi identificado.

Domínio reutilizável

StatusEffectType identifica os efeitos; nesta tarefa contém somente FREEZE. StatusEffectsComponent armazena segundos restantes por tipo em EnumMap, com precisão interna double e consultas float. refresh substitui a duração, advance subtrai dt e remove efeitos expirados, e snapshot retorna cópia imutável. Durações precisam ser finitas e positivas; deltas, finitos e não negativos. Uma tolerância de um microssegundo evita resíduos ao somar passos float de 30 Hz.

StatusEffectService separa elegibilidade e coordenação temporal do armazenamento. apply valida duração, recusa alvos mortos e restringe FREEZE a Mob. MobFactory anexa o componente vazio. Para entidades Mob construídas manualmente por código anterior, apply pode anexar o componente no primeiro sucesso, preservando os testes existentes. Jogadores não recebem Freeze nem têm esse componente anexado pelo serviço.

update avança efeitos existentes e limpa o componente quando o portador morre. Não produz dano e não depende da Criostase. O coordenador do mundo, e não a habilidade, define quando atualizar o relógio. O serviço e os componentes são usados somente pela thread proprietária das entidades.

ProbabilityService recebe java.util.random.RandomGenerator da biblioteca padrão do Java

17. succeeds valida chance finita no intervalo [0,1]. Chance zero sempre falha e chance um sempre sucede, sem consumir o gerador. Chances intermediárias usam uma nova amostra e comparação estrita amostra < chance. Assim, para 30%, 0,29 sucede e 0,30 falha. Amostras inválidas de um gerador customizado também são recusadas.

O serviço é reutilizável sem conhecer mobs, skills ou interface. Não há Math.random no caminho da Criostase. O uso normal recebe RandomGenerator.getDefault; testes podem injetar sequências exatas ou geradores com seed.

Integração da Criostase

CryostasisService mantém seu construtor anterior e acrescenta injeção de StatusEffectService e ProbabilityService. Após combat.attack retornar, verifica dano positivo e sobrevivência. Somente então consome uma amostra de 30% para aquele mob e pulso. Sucesso chama apply(FREEZE,1). O dano continua exclusivamente no CombatService, cujo código e fórmula não foram alterados.

O conjunto já existente de alvos visitados garante uma tentativa no máximo por alvo no mesmo pulso mesmo se a lista contiver duplicatas. Alvos mortos, fora da área, atingidos letalmente ou com dano zero não consomem um sorteio nem recebem novo efeito. Falha de sorteio não chama o serviço de aplicação.

O gerador pertence ao mundo/serviço e não é reinicializado a cada pulso ou mob. DebugWorld permite injetá-lo pelo novo construtor sem perder o construtor padrão. A ordem dos alvos é estável na lista da arena; uma sequência controlada produz resultados reproduzíveis para cada alvo e pulso.

Ordem autoritativa do tick

Após validar dt, o mundo avança os efeitos anteriores dos mobs. Depois atualiza os relógios e processa os pulsos das zonas existentes. Seguem as ações e lançamentos novos, incluindo o pulso imediato. O movimento e o ataque da IA são processados depois disso.

Assim, Freeze aplicado ou renovado por um pulso não sofre outro decremento no mesmo tick, conserva 1 segundo no snapshot e já bloqueia o deslocamento daquele tick. No tick de expiração, se não houver novo sucesso, o movimento volta a ser permitido. Um sucesso no mesmo tick em que o efeito anterior expirou aplica um novo segundo completo.

MovementService consulta FREEZE para mobs antes de resolver o deslocamento. O bloqueio cobre move, moveDirected e moveContinuous. O caminho bloqueado continua validando coordenadas, deslocamentos, overflow e raio antes de retornar. CollisionService permanece geometria sem conhecimento de status; atualizações de posição realizadas fora do serviço não representam ações normais de movimento.

DebugWorld não pula a IA inteira por Freeze. O bloco de ataque e seus cooldowns permanecem ativos, com cálculo de alcance a partir da posição imobilizada. Jogadores não são bloqueados por esse mecanismo.

O passo de simulação permanece limitado a 0,1 s na arena. Em chamadas diretas que cruzem

vários pulsos, todos os pulsos devidos são avaliados na atualização corrente; o coordenador continua responsável por avançar os efeitos antes dos pulsos. Não há reconstrução de movimento contínuo ou de tempos passados de aplicação fora desse modelo de ticks.

Snapshot e interface

DebugSnapshot acrescenta uma lista imutável de StatusView, com ID do ator e duração restante de Freeze. Só mobs vivos com efeito ativo são publicados. ActorView permanece exatamente no contrato anterior; nenhum hash do replay precisou ser recalculado ou alterado.

DebugPanel desenha tonalidade azul-gelo, contorno e legenda CONGELADO a partir dessa lista. Não acessa componentes, sorteia ou avança temporizadores. Reiniciar a arena cria entidades e armazenamento novos, sem reaproveitar efeitos. A fila de Q e os controles anteriores permanecem intactos.

Validação e captura

Comando final: mvn clean test na pasta server. Resultado real: BUILD SUCCESS, compilação release 17, 105 testes, zero falhas, zero erros e zero ignorados. Os 73 testes anteriores continuam aprovados e foram acrescentados 32 testes: 9 do serviço de efeitos, 5 de probabilidades, 7 da integração da Criostase com Freeze, 5 de bloqueio de movimento e 6 da ordem temporal e interface no mundo. Resumo por classe em TASK-005-testes.txt.

Os testes novos cobrem os limites exatos 0%, 30% e 100%, probabilidades inválidas, geradores controlados, amostras 0,29/0,30, armazenamento imutável, duração de 1 segundo, renovação sem acumulação, fracasso preservando o tempo anterior e expiração com passos de 30 Hz.

Também verificam tentativas independentes por alvo e pulso, cadáveres, dano letal e zero, ausência de sexto pulso/dano extra, bloqueio nos caminhos de movimento e retomada após expirar. A integração verifica aplicação antes da IA, segundo completo no tick do pulso, ataque de mob congelado em alcance, ausência de Freeze no jogador, reinício e renderização sem mutação.

Os testes anteriores e os três baselines de MovementReplayTest foram preservados. A captura TASK-005-freeze.png foi produzida pelo teste com amostra controlada 0,29 e inspecionada: exibe o mob congelado, a zona, a legenda, o vigor 45/100 e o cooldown 3 s. O teste também comprova que renderizar não altera o snapshot. Não houve sessão interativa com JFrame nesta etapa.

Arquivos novos e alterados

Novos no domínio, sob server/src/main/java/com/projetok/:
- entity/component/StatusEffectsComponent.java.
- effect/StatusEffectType.java e StatusEffectService.java.
- probability/ProbabilityService.java.

Alterados no domínio:
- factory/MobFactory.java: armazenamento inicial vazio por mob.
- skill/CryostasisService.java: injeção e tentativa após dano positivo sobrevivido.
- movement/MovementService.java: bloqueio de movimento autoritativo.

Alterados no protótipo:
- prototype/DebugWorld.java: relógio de efeitos antes de pulsos e injeção de RNG.
- prototype/DebugSnapshot.java: valores visuais separados do ator legado.
- prototype/DebugPanel.java: tonalidade, contorno e legenda.

Novos testes, sob server/src/test/java/com/projetok/:
- domain/effect/StatusEffectServiceTest.java.
- domain/probability/ProbabilityServiceTest.java.
- domain/skill/CryostasisFreezeTest.java.
- domain/movement/FreezeMovementTest.java.
- prototype/FreezeWorldTest.java.

Documentação e ferramenta:
- README-PROTOTIPO.md e docs/HISTORICO-TECNICO.md.
- docs/TASK-005.md, docs/TASK-005-testes.txt e docs/TASK-005.diff.
- docs/Spirit_Gate_TASK-005.pdf e docs/TASK-005-freeze.png.
- tools/gerar_pdf_task005.py.

Limitações e revisão

Nenhuma dependência foi adicionada. Não há PvP, resistência a controle, imunidade, Freeze em jogadores, novas skills ou alteração de balanceamento da Criostase. O sistema permite estender tipos de efeitos, mas só FREEZE foi implementado. O armazenamento não tem regras de habilidade; o chamador de gameplay deve usar StatusEffectService para elegibilidade.

A aleatoriedade real varia entre sessões; reproduzir uma sessão exige injetar o mesmo gerador/seed e a mesma ordem de comandos e alvos. Testes usam sequências explícitas, não testes estatísticos sujeitos a flutuação.

Esta pasta não é um checkout Git acessível; não houve commit, merge ou integração em branch. O diff textual é relativo ao início desta tarefa, sem server/target, binários temporários ou inclusão recursiva de si mesmo. PDF e captura são entregas separadas de documentação.

Reproduzir: mvn clean test na pasta server; python -B tools/gerar_pdf_task005.py na pasta Projeto-K para regenerar o PDF. Conferência manual: lançar Q sobre mobs e observar que os congelados param de perseguir, mantêm ataques próximos e retomam o movimento ao expirar. Como a chance é de 30%, nem todo pulso congela.